

Benefits and Use case of serverless cost efficiency

Tasks-1.List 3 real-world apps you use (like Zomato, Instagram, or Paytm) and describe one scenario in each where using serverless architecture would improve cost efficiency or scalability.

App	Serverless use case	Benefit
Instagram	Generate image-processing jobs when users upload photos/videos. Lambda can process thumbnails or metadata only when needed.	No servers need to run continuously; compute scales with uploads, reducing idle costs.
Zomato	Handle restaurant-order notifications and status updates using event-triggered Lambda functions.	Lambda automatically scales during lunch/dinner peaks and reduces infrastructure costs during quiet periods.
Paytm	Process transaction notifications or payment-status events asynchronously.	Serverless functions can scale automatically during transaction spikes without maintaining a large always-on server fleet.

Tasks-2.Create a simple REST API for a 'Favorite Songs' list using AWS Lambda and API Gateway — implement a Lambda function in your preferred language that returns a hardcoded list of 3 song names when accessed via an HTTP GET request.

Create a Lambda function called:

FavoriteSongsAPI

Use this code:

```
import json
```

```
songs = [
```

```
    "Blinding Lights - The Weeknd",
```

```
    "Shape of You - Ed Sheeran",
```

```
    "Believer - Imagine Dragons"
```

```
]
```

```
def lambda_handler(event, context):  
    return {  
        "statusCode": 200,  
        "headers": {  
            "Content-Type": "application/json"  
        },  
        "body": json.dumps(songs)  
    }
```

Click **Deploy**.

Create the API Gateway endpoint

In AWS:

API Gateway → Create API → HTTP API

Configure the integration to your FavoriteSongsAPI Lambda function.

Create a **GET** route:

GET /songs

After creating the API, API Gateway will provide an **Invoke URL**, for example:

<https://abc123.execute-api.ap-south-1.amazonaws.com>

Your endpoint will then be:

GET <https://abc123.execute-api.ap-south-1.amazonaws.com/songs>

Tasks-3. Modify your Lambda function to accept a POST request that adds a new song name (sent in the request body) to an in-memory array, and returns the updated list. Use a global variable in your Lambda function to simulate the array during the same execution (stateless between invocations).

Now modify the Lambda so it handles both **GET** and **POST**.

```
import json
```

```
# Global variable used to simulate an in-memory array.
```

```
# It may persist between warm Lambda invocations,
```

```
# but it is NOT permanent storage.
```

```
songs = [
```

```
    "Blinding Lights - The Weeknd",
```

```
    "Shape of You - Ed Sheeran",
```

```
    "Believer - Imagine Dragons"
```

```
]
```

```
def lambda_handler(event, context):
```

```
    method = event.get("requestContext", {}).get("http", {}).get("method")
```

```
    # GET /songs
```

```
    if method == "GET":
```

```
        return {
```

```
            "statusCode": 200,
```

```
            "headers": {
```

```
                "Content-Type": "application/json"
```

```
            },
```

```
            "body": json.dumps(songs)
```

```
        }
```

```
# POST /songs

elif method == "POST":

    try:

        body = event.get("body", "{}")

        if isinstance(body, str):

            body = json.loads(body)

        new_song = body.get("song")

        if not new_song:

            return {

                "statusCode": 400,

                "headers": {

                    "Content-Type": "application/json"

                },

                "body": json.dumps({

                    "error": "Please provide a song name"

                })

            }

        songs.append(new_song)

    return {

        "statusCode": 200,
```

```
    "headers": {  
        "Content-Type": "application/json"  
    },  
    "body": json.dumps(songs)  
}
```

except Exception as e:

```
    return {  
        "statusCode": 400,  
        "body": json.dumps({  
            "error": str(e)  
        })  
    }
```

```
return {  
    "statusCode": 405,  
    "body": json.dumps({  
        "error": "Method not allowed"  
    })  
}
```

Click **Deploy**.

Tasks-4. Configure an API Gateway endpoint to trigger your Lambda function for both GET and POST requests, and test both endpoints using Postman or curl. Make sure to set the correct HTTP method mappings in API Gateway.

In **API Gateway**, configure both routes:

GET /songs

POST /songs

[
"Blinding Lights - The Weeknd",

```
"Shape of You - Ed Sheeran",  
"Believer - Imagine Dragons",  
"Perfect - Ed Sheeran"
```

```
]
```

Then perform another GET:

```
curl https://YOUR_API_ID.execute-api.ap-south-1.amazonaws.com/songs
```

If the same Lambda execution environment is reused, you should see the newly added song.

Postman

For **GET**:

- Method: GET
- URL: `https://YOUR_API_ID.execute-api.ap-south-1.amazonaws.com/songs`
- Click **Send**

For **POST**:

- Method: POST
- Same URL
- **Body → raw → JSON**
- Enter:

```
{  
  
  "song": "Perfect - Ed Sheeran"  
}
```

Click **Send**.

Tasks-5. Pick a real-time data processing use case from an app you use (for example, live IPL score updates in Cricbuzz or real-time chat in WhatsApp). Explain in 3-4 lines how you would design a serverless workflow for this using Lambda, API Gateway, and any other AWS service if needed.

Example: Cricbuzz live IPL scores

A cricket-scoring system could send score updates through **API Gateway** to Lambda whenever a ball or scoring event occurs. Lambda validates the update and stores the latest match state in **DynamoDB**. Clients can receive updates through a WebSocket/API mechanism, while **EventBridge** or other event services can distribute important events such as wickets or innings completion.

Workflow:

Cricket scoring app



API Gateway



AWS Lambda



DynamoDB



Live clients / notifications